DiffPower.jl: A Differentiable Modeling Framework in Julia for Simulation, Analysis, and Optimization of Large-Scale Power Systems

Aizuo Chen*†, Jiaxiang Hu*†, Xiangyu Meng*†, and Yue Zhu*†
* Department of Electrical Engineering, City University of Hong Kong, Hong Kong SAR, China
† City University of Hong Kong Shenzhen Research Institute, Shenzhen, China
Email: aizuochen2-c@my.cityu.edu.hk, jiaxianhu2-c@my.cityu.edu.hk, xmeng76@cityu.edu.hk, yue.zhu@cityu.edu.hk

***Abstract*—This paper presents DiffPower.jl, an open-source Julia package that provides a differentiable modeling framework for simulation, analysis, and parameter optimization of large-scale power systems. In DiffPower.jl, buses, lines, and dynamic devices are modeled as modular differentiable components and automatically composed into a unified nonlinear ordinary-differential-equation representation. The same assembled model is used consistently for time-domain simulation, automatic linearization, modal analysis, participation-factor analysis, parameter sensitivity analysis, and stability-oriented parameter tuning. By leveraging the Julia SciML ecosystem, automatic differentiation, sparse Jacobian construction, graph coloring, and high-performance sparse linear algebra, DiffPower.jl reduces the need for manually maintained Jacobians while enabling scalable electromagnetic-transient- level studies. An IEEE 300-bus workflow demonstrates the package's capabilities in large-scale differentiable model construction, equilibrium diagnostics, sparse Jacobian evaluation, eigenvalue-based stability assessment, sensitivity-guided parameter optimization, and dynamic simulation.**

***Index Terms*—differentiable modeling, electromagnetic transient simulation, modal analysis, sensitivity analysis, parameter optimization, Julia, automatic differentiation, sparse Jacobian.**

I. Introduction

The increasing penetration of inverter-based resources has fundamentally reshaped the dynamic behavior of modern power systems [1]. Compared with conventional electromechanical transient models, electromagnetic-transient (EMT)-level models can represent fast voltage- and current-control dynamics, phase-locked-loop dynamics, grid-forming control characteristics, and electromagnetic network dynamics. These capabilities are essential for analyzing weak grids, converter-dominated systems, high-frequency oscillations, and dynamic interactions among synchronous machines, inverter-based resources, and transmission networks.

However, large-scale EMT-level simulation, analysis, and optimization remain computationally challenging [2]. Practical systems may contain hundreds of buses, transmission branches, and heterogeneous dynamic devices, leading to stiff, high-dimensional, sparse, and strongly coupled dynamic models. Beyond time-domain simulation, small-signal modal analysis, participation-factor analysis, parameter sensitivity analysis, and stability-oriented tuning all require Jacobians of the complete dynamic system around an operating point. Dense Jacobian construction can be expensive, while manually deriving and maintaining analytical Jacobians becomes impractical as device libraries and controller models evolve.

Existing power system tools have made important contributions in their respective domains. MATPOWER and PowerModels.jl are widely used for steady-state power-flow and optimal-power-flow studies [3], [4]. MATLAB/Simulink and PSCAD provide mature simulation environments for dynamic and EMT studies. Nevertheless, these simulation-oriented environments are generally not designed as automatic-differentiation-native modeling frameworks. As a result, linearization, sensitivity analysis, and parameter optimization often rely on tool-specific workflows, numerical perturbation, separately maintained linearization routines, or manually derived Jacobians. This separation between the nonlinear simulation model and the analysis or optimization model can increase implementation effort and may reduce consistency, extensibility, and reproducibility in large-scale converter-dominated system studies.

A differentiable modeling framework provides a natural way to address these limitations. By representing buses, transmission lines, and dynamic devices as modular differentiable components, the complete power system can be automatically composed into a unified nonlinear ordinary differential equation representation. The same assembled model can then be used consistently for time-domain simulation, automatic linearization, modal analysis, participation-factor analysis, parameter sensitivity analysis, and stability-oriented parameter optimization. This design reduces the need for manually maintained Jacobians and provides a direct connection between component-level equations and system-level stability metrics.

Julia provides an attractive foundation for implementing such a framework. Its high-level syntax, just-in-time compilation, multiple dispatch, and ecosystem for differential equations, automatic differentiation, sparse linear algebra, and optimization enable clear model implementation while preserving high computational performance [5], [6]. In particular, ForwardDiff.jl, SparseDiffTools.jl, and SparseMatrixColorings.jl support automatic differentiation, sparsity detection, graph coloring, and sparse Jacobian reconstruction [7]–[9]. Since power system Jacobians are typically sparse, graph coloring can reduce the number of derivative directions from the number of states to the number of structurally independent

column groups [10], [11].

To this end, this paper introduces *DiffPower.jl*, an open-source Julia package for differentiable modeling, simulation, analysis, and parameter optimization of large-scale power systems. DiffPower.jl models buses, transmission lines, and dynamic devices as modular differentiable components, and automatically composes them into a unified nonlinear dynamic model. The main contributions are summarized as follows:

- A differentiable modeling framework is developed for large-scale power system dynamic simulation, analysis, and parameter optimization.
- A modular component architecture is implemented for buses, transmission lines, and heterogeneous dynamic devices, including synchronous generators, grid-forming inverters, and grid-following inverters.
- A compositional workflow is provided to automatically assemble global state vectors, parameter vectors, initial conditions, state maps, parameter maps, and right-hand-side functions from power-flow data and component definitions.
- Dense and sparse automatic-differentiation-based Jacobian tools are integrated with graph coloring and sparse matrix structures for scalable linearization, modal analysis, sensitivity computation, and time-domain simulation.
- A sensitivity-guided spectral parameter optimization procedure is developed to rank candidate parameters, evaluate bounded trial updates, and reduce the spectral abscissa of the linearized dynamic system.
- An IEEE 300-bus case study demonstrates differentiable model construction, equilibrium diagnostics, sparse Jacobian evaluation, eigenvalue and participation-factor analysis, parameter optimization, EMT simulation, and sparse Jacobian performance evaluation.

II. DiffPower.jl Package Architecture

A. Overall Structure

DiffPower.jl is designed as a differentiable modeling framework for large-scale power system dynamic studies. Its architecture consists of three main layers: differentiable component modeling, compositional system assembly, and simulation-analysis-optimization utilities. The component modeling layer defines reusable differentiable models for buses, transmission lines, and dynamic devices. The system assembly layer composes local component equations into a unified nonlinear ordinary differential equation model. The utility layer provides numerical tools for time-domain simulation, automatic linearization, modal analysis, participation-factor analysis, parameter sensitivity analysis, and stability-oriented parameter optimization.

The current package includes the following main modules:

- `Node.jl`: bus voltage dynamics, nodal current-balance equations, and node-level parameters;
- `Line.jl`: dynamic transmission-line models, branch-current states, and endpoint mapping;
- `Devices.jl`: dynamic device models, including synchronous generators, infinite-bus models, grid-forming inverters, and grid-following inverters;
- `composing.jl`: global model assembly, state maps, parameter maps, component ownership records, and system right-hand-side construction;
- `generating.jl`: automatic component generation from power-flow data;
- `residual_diagnostics.jl`: equilibrium checking and initialization residual diagnostics;
- `observing.jl`: observation registration, output maps, solution-level measurement extraction, and event-aware observation under parameter changes;
- `modal_diagnostics.jl`: Jacobian construction, eigenvalue computation, damping analysis, and participation-factor analysis;
- `spectral_param_sensitivity.jl`: spectral sensitivity calculation, parameter ranking, and stability-oriented parameter optimization.

Fig. 1 shows the conceptual architecture of DiffPower.jl.

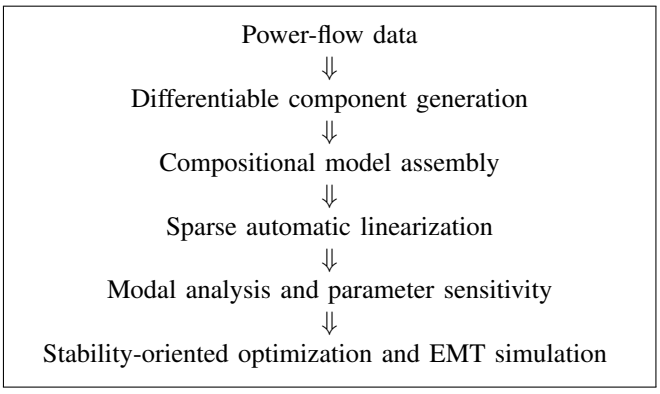


Fig. 1. Conceptual architecture of the DiffPower.jl package.

B. Differentiable Component Interface

Each physical component in DiffPower.jl is represented by a Julia type and a set of interface functions. Node models provide state definitions, parameter definitions, initial conditions, and bus voltage dynamics. Line models provide endpoint information, branch-current states, branch parameters, and branch differential equations. Device models provide local states, local parameters, current-injection equations, dynamic equations, initialization rules, and optional observation functions.

This interface-based design separates local component equations from global system assembly. Therefore, new generator models, inverter controllers, reference devices, or other dynamic components can be added by implementing the required component interface, without modifying the global solver, automatic differentiation routines, modal-analysis tools, or optimization procedures. Since the component equations are implemented as differentiable Julia functions, the assembled system model can be used directly for both nonlinear simulation and derivative-based analysis.

C. Compositional System Assembly

DiffPower.jl uses state labels and parameter labels to construct global vectors from local component definitions. Each state is associated with a component type, component index, and symbolic variable name. Similarly, each parameter is recorded with its component ownership information. The global state vector is written as

$$x = \begin{bmatrix} x_{\text{node}}^T & x_{\text{line}}^T & x_{\text{dev}}^T \end{bmatrix}^T, \tag{1}$$

where x_{node}, x_{line}, and x_{dev} are the bus, transmission-line, and device state vectors, respectively.

The global parameter vector is written as

$$\theta = \begin{bmatrix} \theta_{\text{node}}^T & \theta_{\text{line}}^T & \theta_{\text{dev}}^T \end{bmatrix}^T. \tag{2}$$

Based on these maps, local component equations are automatically composed into a global nonlinear dynamic model,

$$\dot{x} = F(x, \theta, t). \tag{3}$$

The same assembled function F is used for time-domain simulation, automatic Jacobian construction, modal analysis, parameter sensitivity analysis, and stability-oriented parameter optimization. This label-based representation provides a direct connection between numerical indices and physical variables, which is useful for debugging, initialization diagnostics, modal interpretation, participation-factor analysis, and parameter-sensitivity studies.

III. Differentiable Modeling and Analysis Formulation

A. Unified Dynamic Representation

The assembled power system model in DiffPower.jl is represented as a nonlinear ordinary differential equation:

$$\dot{x} = F(x, \theta, t), \tag{4}$$

where x is the global state vector, θ is the parameter vector, and t is time. The right-hand-side function F is automatically composed from differentiable node, line, and device components.

For bus i, the node dynamics are written in the general form

$$\dot{x}_{n,i} = f_{n,i}\left(x_{n,i}, i_{d,i}^{\text{inj}}, i_{q,i}^{\text{inj}}, \theta_{n,i}\right), \tag{5}$$

where $i_{d,i}^{\text{inj}}$ and $i_{q,i}^{\text{inj}}$ are the total injected currents in the rotating reference frame.

For line ℓ connecting buses i and j, the branch dynamics are given by

$$\dot{x}_\ell = f_\ell\left(x_\ell, v_i, v_j, \theta_\ell\right), \tag{6}$$

where v_i and v_j are the terminal bus voltage vectors.

For dynamic device k connected to bus i, the local dynamics and current injection are represented by

$$\dot{x}_{d,k} = f_{d,k}\left(x_{d,k}, v_i, \theta_{d,k}\right), \tag{7}$$

$$i_{d,k}^{\text{out}} = g_{d,k}\left(x_{d,k}, v_i, \theta_{d,k}\right). \tag{8}$$

The global function F aggregates all node, line, and device contributions. Since these component equations are implemented as differentiable Julia functions, the same assembled model can be used for time-domain simulation, automatic linearization, sensitivity analysis, and parameter optimization.

B. Equilibrium Initialization and Residual Diagnostics

Given a solved power-flow case, DiffPower.jl generates component parameters and initial states that are consistent with the operating point. The equilibrium condition is

$$F(x_0, \theta_0, 0) = 0. \tag{9}$$

In practice, residual diagnostics are used to evaluate the initialization quality:

$$r_0 = \left\| F(x_0, \theta_0, 0) \right\|_\infty. \tag{10}$$

A small value of r_0 indicates that the assembled dynamic model is consistent with the steady-state operating point.

C. Automatic Linearization and Modal Analysis

Small-signal modal analysis is performed by differentiating (4) around the equilibrium point:

$$\Delta\dot{x} = A\Delta x, \tag{11}$$

where

$$A = \left.\frac{\partial F}{\partial x}\right|_{x=x_0, \theta=\theta_0}. \tag{12}$$

In DiffPower.jl, A is constructed directly from the same differentiable function F used for nonlinear simulation. This avoids maintaining a separate analytical linearization model.

The eigenvalues and right eigenvectors satisfy

$$Av_i = \lambda_i v_i. \tag{13}$$

For $\lambda_i = \sigma_i + j\omega_i$, the oscillation frequency and damping ratio are

$$f_i = \frac{|\omega_i|}{2\pi}, \qquad \zeta_i = \frac{-\sigma_i}{\sqrt{\sigma_i^2 + \omega_i^2}}. \tag{14}$$

The linearized system is locally asymptotically stable if all eigenvalues have negative real parts.

D. Participation Factors and Parameter Sensitivity

Let w_i be the left eigenvector associated with λ_i, satisfying

$$w_i^T A = \lambda_i w_i^T. \tag{15}$$

The participation factor of state k in mode i is

$$p_{k,i} = v_{k,i} w_{i,k}. \tag{16}$$

DiffPower.jl maps state-level participation factors back to physical components, which helps identify the buses, lines, and devices most strongly associated with a critical mode.

For a parameter θ_j, the eigenvalue sensitivity is computed as

$$\frac{d\lambda_i}{d\theta_j} = \frac{w_i^T \left(\dfrac{dA}{d\theta_j}\right) v_i}{w_i^T v_i}. \tag{17}$$

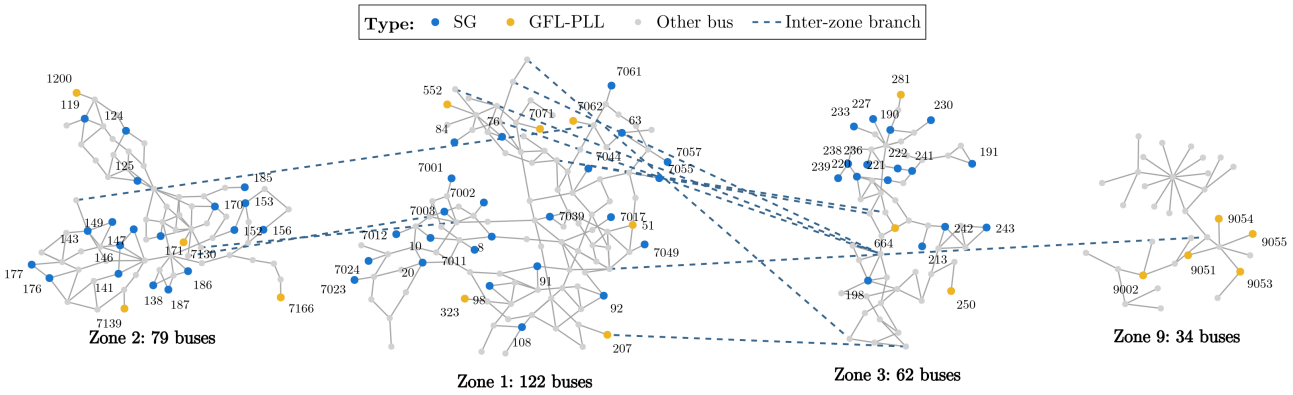


Fig. 2. Topology and device distribution of the IEEE 300-bus test system.

When a parameter is updated, the equilibrium-dependent quantities and the linearized matrix are recomputed from the same assembled model. This ensures consistency between parameter changes, linearization, and modal analysis.

E. Spectral Parameter Optimization

The stability-oriented optimization metric is the spectral abscissa:

$$\alpha(\theta) = \max_i \operatorname{Re}\left[\lambda_i\big(A(\theta)\big)\right]. \tag{18}$$

A smaller spectral abscissa indicates improved small-signal stability. DiffPower.jl ranks candidate parameters according to their spectral sensitivities, evaluates bounded trial updates with multiple step sizes, and accepts the update that produces the largest reduction in $\alpha(\theta)$.

F. Sparse Automatic Differentiation

For large-scale power system models, the Jacobian matrix is typically highly sparse. DiffPower.jl exploits this sparsity by combining automatic differentiation with precomputed sparsity patterns and graph coloring. Structurally independent columns are grouped and evaluated together, reducing the number of derivative directions required for Jacobian construction.

The package supports dense automatic differentiation for small systems and sparse automatic differentiation for large-scale cases, such as the IEEE 300-bus EMT-level model studied in this paper.

IV. Case Study: IEEE 300-Bus System

A. Study Workflow

An IEEE 300-bus system is used to demonstrate the workflow of DiffPower.jl for large-scale differentiable modeling, modal analysis, parameter optimization, and time-domain simulation. The system contains 300 buses, 408 transmission lines, 59 synchronous generators, and 18 grid-following inverters. Its network topology and device distribution are shown in Fig. 2.

Starting from power-flow data, DiffPower.jl generates differentiable bus, line, and device components, assembles the global nonlinear dynamic model, constructs the initial condition, checks equilibrium residuals, computes AD-based Jacobians, performs modal and sensitivity analysis, tunes selected parameters, and runs EMT simulation.

1: Load the IEEE 300-bus power-flow data.
2: Generate differentiable bus, line, and device components.
3: Assemble the global state map, parameter map, and dynamic model.
4: Construct the initial condition and check equilibrium residuals.
5: Compute dense or sparse AD-based Jacobians.
6: Perform modal and participation-factor analysis.
7: Conduct sensitivity-guided parameter optimization.
8: Run EMT simulation and observe selected outputs.

Fig. 3. DiffPower.jl workflow for the IEEE 300-bus case study.

A conceptual Julia API workflow is shown in Fig. 4. The same assembled differentiable model is used consistently for time-domain simulation, automatic linearization, modal analysis, sensitivity analysis, and parameter optimization.

B. Modal Analysis and Parameter Optimization

After equilibrium initialization, the Jacobian matrix of the IEEE 300-bus EMT-level model was evaluated at the operating point. Before parameter tuning, the dominant eigenvalue is

$$\lambda_1 = 1.7889 + j\,4474.5924. \tag{19}$$

Its positive real part indicates small-signal instability. The corresponding oscillation frequency is

$$f_1 = \frac{4474.5924}{2\pi} = 712.15 \text{ Hz}, \tag{20}$$

showing that the critical mode is a high-frequency EMT mode.

TABLE I
Dominant Global Mode Before Parameter Tuning

Quantity	Value
Eigenvalue λ_1	$1.7889 + j\,4474.5924$
Spectral abscissa	1.7889
Oscillation frequency	712.15 Hz
Damping ratio	-3.998×10^{-4}

`generate_components_from_pf` → `build_model` → `build_x0` → `system_rhs!` → `prepare_sparse_ad` → `modal_analysis` → `spectral_sensitivity` → `parameter_optimization` → `solve` → `observe_solution`.

Fig. 4. Conceptual Julia API workflow used by DiffPower.jl.

Participation-factor analysis shows that the unstable mode is mainly associated with GFL-PLL device 69 at bus 9055 and line 9 between buses 9005 and 9055. Their participation factors are 53.8455% and 41.0533%, respectively, accounting for about 94.90% of the total participation.

TABLE II
MAIN COMPONENT PARTICIPATION FACTORS BEFORE PARAMETER TUNING

Rank	Component	Participation
1	GFL-PLL device 69, bus 9055	53.8455%
2	Line 9, 9005–9055	41.0533%
3	GFL-PLL device 68, bus 9054	2.2753%
4	Line 8, 9005–9054	2.0009%

Based on this result, the filter capacitance c_f of GFL-PLL device 69 was selected for stability-oriented optimization. Starting from $c_f = 0.0090145$, the optimization routine returned $c_f = 0.0045072$. The spectral abscissa decreased from 1.7889 to -3.4785×10^{-4}, indicating that the optimized linearized model is small-signal stable.

C. Time-Domain EMT Simulation

Time-domain EMT simulation was performed using the same assembled model used for modal analysis and parameter optimization. A sparse AD-based Jacobian was attached to the ODE function to support stiff integration. The simulation interval was 0–200 s, using `KenCarp4` with a KLU sparse linear solver and relative and absolute tolerances of 10^{-8}. A parameter disturbance was applied to device 61 by changing its reference active power p_r at $t = 0.1$ s through a discrete callback.

The recorded outputs include bus voltage magnitudes, synchronous-generator rotor speeds, and the active-power response of device 61. These quantities were obtained through the observation interface of DiffPower.jl.

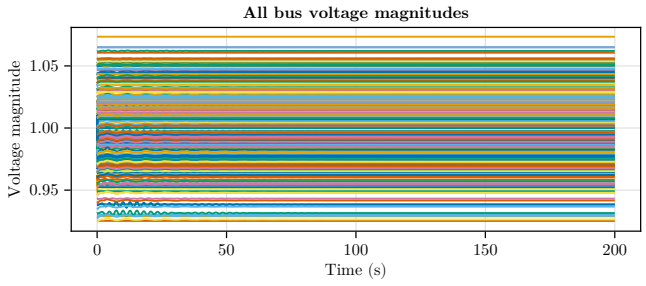


Fig. 5. Voltage magnitude trajectories of all buses under the device-parameter disturbance.

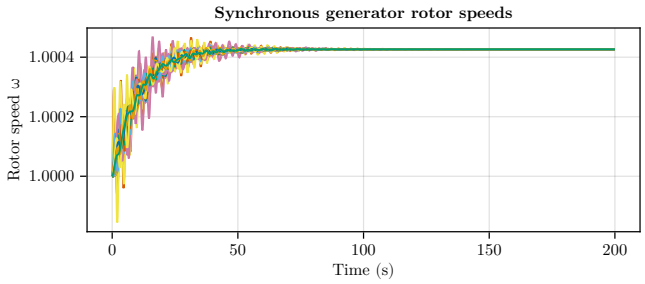


Fig. 6. Rotor speed trajectories of synchronous generators after the disturbance.

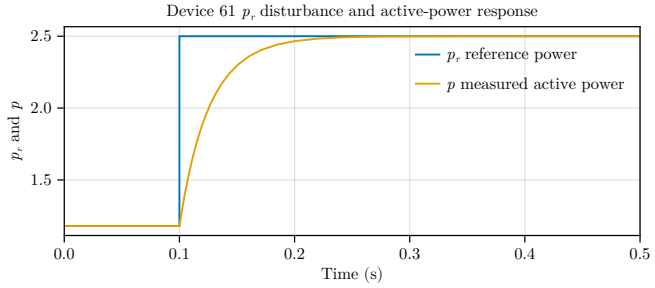


Fig. 7. Reference active-power disturbance and measured active-power response of device 61.

D. Sparse Jacobian Performance

Table III reports the Jacobian construction performance for the IEEE 300-bus EMT-level model. The sparse Jacobian contains 9977 stored entries out of 1861^2 possible entries, corresponding to a density of 0.2881%.

TABLE III
MEASURED JACOBIAN CONSTRUCTION PERFORMANCE

Metric	Value
Number of buses	300
Total number of states	1861
Jacobian matrix size	1861×1861
Sparse Jacobian stored entries	9977
Jacobian density	0.2881%
Dense Jacobian time	25.643 ms
Sparse Jacobian time	307.100 μs
Sparse Jacobian speedup	$83.50\times$
Dense Jacobian allocations	591
Sparse Jacobian allocations	11
Dense Jacobian memory allocation	55.43 MiB
Sparse Jacobian memory allocation	170.80 KiB

Compared with dense automatic differentiation, sparse automatic differentiation reduces the Jacobian construction time

from 25.643 ms to 307.100 μs, achieving an $83.50\times$ speedup. Memory allocation is reduced from 55.43 MiB to 170.80 KiB.

V. Conclusion

This paper presented DiffPower.jl, an open-source Julia package for differentiable modeling, simulation, analysis, and parameter optimization of large-scale power systems. DiffPower.jl represents buses, transmission lines, and dynamic devices as modular differentiable components and automatically composes them into a unified nonlinear dynamic model. The same assembled model is used consistently for power-flow-based initialization, equilibrium diagnostics, time-domain EMT simulation, automatic Jacobian construction, modal analysis, participation-factor analysis, parameter sensitivity analysis, and stability-oriented parameter optimization.

The proposed differentiable modeling framework provides several practical advantages for large-scale dynamic studies. Its component-based design improves extensibility, allowing new generator models, inverter controllers, load models, and protection models to be added through local component interfaces. Automatic differentiation reduces the need for manually derived and separately maintained Jacobians, while sparse matrix structures and graph coloring improve the scalability of Jacobian construction. These features are particularly useful for converter-dominated systems, where device models and controller structures may evolve frequently.

The IEEE 300-bus case study demonstrated the complete workflow of DiffPower.jl. The package identified a high-frequency unstable mode, localized the dominant participating components, guided effective parameter tuning, and stabilized the linearized model. The sparse automatic-differentiation workflow reduced Jacobian construction time from 25.643 ms to 307.100 μs, achieving an $83.50\times$ speedup, while memory allocation was reduced from 55.43 MiB to 170.80 KiB. These results demonstrate that DiffPower.jl provides a transparent, extensible, and computationally efficient differentiable modeling platform for large-scale power system dynamic studies.

Future work will focus on improving reverse-mode differentiation support, extending constrained optimization capabilities, expanding the dynamic device library, and applying the package to learning-based stability assessment and control problems.

Acknowledgment

This work was supported by the Smart Grid National Science and Technology Major Project under Grant No. 2026ZD0809800.

References

[1] A. Chen, Y. Zhu, and H. Zhao, "Almost globally stable gfl inverter control scheme based on current control synchronization," *IEEE Transactions on Sustainable Energy*, vol. 17, no. 3, pp. 2926–2938, 2026.

[2] Y. Zhu, Y. Gu, Y. Li, and T. C. Green, "Impedance-based root-cause analysis: Comparative study of impedance models and calculation of eigenvalue sensitivity," *IEEE Transactions on Power Systems*, vol. 38, no. 2, pp. 1642–1654, Mar. 2023.

[3] R. D. Zimmerman, C. E. Murillo-Sanchez, and R. J. Thomas, "MATPOWER: Steady-state operations, planning, and analysis tools for power systems research and education," *IEEE Transactions on Power Systems*, vol. 26, no. 1, pp. 12–19, Feb. 2011.

[4] C. Coffrin, R. Bent, K. Sundar, Y. Ng, and M. Lubin, "PowerModels.jl: An open-source framework for exploring power flow formulations," in *Proc. Power Systems Computation Conference*, Dublin, Ireland, 2018, pp. 1–8.

[5] J. Bezanson, A. Edelman, S. Karpinski, and V. B. Shah, "Julia: A fresh approach to numerical computing," *SIAM Review*, vol. 59, no. 1, pp. 65–98, 2017.

[6] C. Rackauckas and Q. Nie, "DifferentialEquations.jl–a performant and feature-rich ecosystem for solving differential equations in Julia," *Journal of Open Research Software*, vol. 5, no. 1, 2017.

[7] J. Revels, M. Lubin, and T. Papamarkou, "Forward-mode automatic differentiation in Julia," 2016, arXiv:1607.07892.

[8] SciML, "SparseDiffTools.jl: Sparse automatic differentiation and matrix coloring tools in Julia," [Online]. Available: https://docs.sciml.ai/SparseDiffTools/, accessed: Jun. 23, 2026.

[9] A. Montoison, G. Dalle, and A. H. Gebremedhin, "Revisiting sparse matrix coloring and bicoloring," 2025, arXiv:2505.07308.

[10] A. R. Curtis, M. J. D. Powell, and J. K. Reid, "On the estimation of sparse Jacobian matrices," *Journal of the Institute of Mathematics and Its Applications*, vol. 13, no. 1, pp. 117–119, 1974.

[11] A. H. Gebremedhin, F. Manne, and A. Pothen, "What color is your Jacobian? graph coloring for computing derivatives," *SIAM Review*, vol. 47, no. 4, pp. 629–705, 2005.